

**Department of Statistics and Computer Science**  
**University of Peradeniya**  
**CSC2021 – Programming using Data Structures**  
**(2020/21)**  
**Labsheet 05**  
**Hash Tables**

---

**Purpose**

- Understanding the Hash tables data structure and its implementation
- Learn what is a collision in a hash table and various types of collision resolution methods available.

**Theory**

- Both in arrays and linked lists retrieving a specific element (searching and retrieving) is mainly done using a brute force approach (Going through the array/linked list starting from the beginning and searching through each element until the necessary element is found or arrive at the end of the array.)
- This may take a lot of time especially if there's a lot of data, worst case scenario having a time complexity of  $O(n)$ .
- A hash table is a data structure that allows fast retrieval of data with a constant time complexity (  $O(1)$  ) in an ideal setting.
- A hash table uses a special key (can be the element itself or part of the element) which is converted into a specific index called hash value that can be used for insertion as well as retrieval of data.
- The method that is used to generate this hash value is called the hash function.
- Hence a hash table has two main parts,
  - The data structure that is used to store the elements (usually this is an array).
  - The hash function.
- Hash Function
  - A hash function is a method that can be used to convert an element (if element is a single component like an integer value) or part of the element (if the element consists of complex components like an object) into a simple value that can be used as an index to access the array.

- A good hash function should have the following characteristics,
  - Generation of the hash code should be simple and must not become an algorithm in itself
  - It should provide a uniform distribution across the hash table (elements should be distributed uniformly across the table and should not be cluttered in one location).
  - Should result in less amount of collisions.
- Example hash table (array size = 10)

- Elements = abc, def, ijk, bcd
- Hash function -> sum of ASCII values of the characters % size of the table (array size)
  - $abc = (97 + 98 + 99) \% 10 \Rightarrow 9$
  - $def = (100 + 101 + 102) \% 10 \Rightarrow 3$
  - $ghi = (103 + 104 + 105) \% 10 \Rightarrow 4$
  - $bcd = (98 + 99 + 100) \% 10 = 7$

|  |  |  |     |     |  |  |     |  |     |
|--|--|--|-----|-----|--|--|-----|--|-----|
|  |  |  | def | ghi |  |  | bcd |  | abc |
|--|--|--|-----|-----|--|--|-----|--|-----|

- A collision is an instance where two different elements generate the same hash value. Hence they are to be inserted in the same location.
- Collision Example:

- Insertion of value “aiy”

$$aiy = (97 + 105 + 121) \% 10 \Rightarrow 3$$

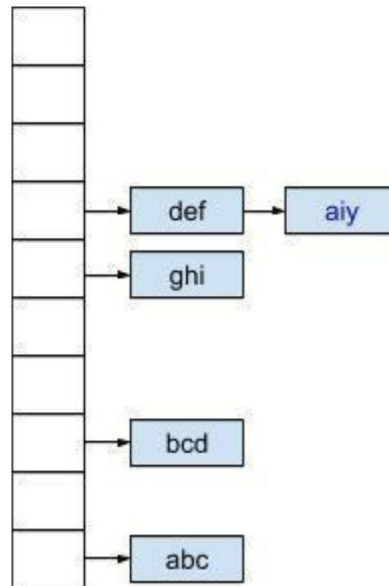
aiy

|  |  |  |     |     |  |  |     |  |     |
|--|--|--|-----|-----|--|--|-----|--|-----|
|  |  |  | def | ghi |  |  | bcd |  | abc |
|--|--|--|-----|-----|--|--|-----|--|-----|

- There are several different collision resolution techniques
  - **Linear Probing/ Open addressing** - insert the element into the next free index.

|  |  |  |     |     |     |  |     |  |     |
|--|--|--|-----|-----|-----|--|-----|--|-----|
|  |  |  | def | ghi | aiy |  | bcd |  | abc |
|--|--|--|-----|-----|-----|--|-----|--|-----|

- **Separate Chaining** - Each hash value consists of a linked list, elements with similar hash value are inserted to the same linked list.



- **Quadratic Probing** - Similar to linear probing but the next free slot is checked by adding a quadratic constant to the current index
  - Index = hash code = i,
  - If i is occupied, index =  $i + 1^2 (\% \text{hashTableSize})$
  - If  $i + 1^2$  is occupied, index =  $i + 2^2 (\% \text{hashTableSize})$ , etc

|  |  |     |     |     |  |  |     |  |     |
|--|--|-----|-----|-----|--|--|-----|--|-----|
|  |  | aiy | def | ghi |  |  | bcd |  | abc |
|--|--|-----|-----|-----|--|--|-----|--|-----|

- **Double Hashing** - The next iteration of the hash value is found by another new hash function.
  - Index = hashfunc\_1(value) = hash code = i
  - If index is occupied, index =  $(i + 1 * \text{hashfunc}_2(\text{value})) (\% \text{hashTableSize})$
  - If index is occupied, index =  $(i + 2 * \text{hashfunc}_2(\text{value})) (\% \text{hashTableSize})$ , etc

- Whenever a collision resolution technique is used the retrieval process must also be changed accordingly

(eg: For retrieval for a hashtable having linear probing collision resolution, a comparison should be made with the element existing at the location of the obtained hash code, if the element is not similar, a continuous linear probing should be carried out until the necessary element is found.)

### **Question 01**

- Implement a hash table that can be used to store a list of numbers.
  - Assume that this is used to store up to maximum of 5 indexes.
  - Use modulo by 5 as the hash function.
  - Use separate chaining as the collision resolution technique.
  - Insert the following numbers to hash table.
    - 5,10,20,1,0,100,13,21
  - Print the hash table in the existing order of the elements.
  - Remove number 100 from hash table.
  - Print again all the numbers in hash table.

### **Question 02**

- Implement a hash table to store extended details of the books mentioned above
  - Use separate chaining as the collision resolution technique.
  - Insert the following data , use 'author' in order to generate hash code.  
(Order - book\_id, book\_name, author, num\_pages, ratings\_count)  
  
B01, Harry Potter, Rowling, 652, 2095690  
B02, The Lost Continent, Bryson, 299, 45712  
B03, Notes from a Small Island, Bryson, 324, 80609  
B04, The Lord of the Rings, Tolkien, 1184,1710  
B05, The Changeling Sea,Patricia, 137,4454  
B06, Heirs of General Practice, McPhee, 128,268  
B07, Salmon of Doubt, Douglas, 336,5  
B08, For the New Intellectual, Rand, 224,2750  
B09, City of Glass, Auster, 203,12410  
B10, A War Like No Other, Davis, 397,1693
  - Print the data in the hash table in the existing order of the elements.
  - Implement retrieve function to retrieve "The Loard of the Rings" from hash table.